

TP2 — Stateful Web Tracking

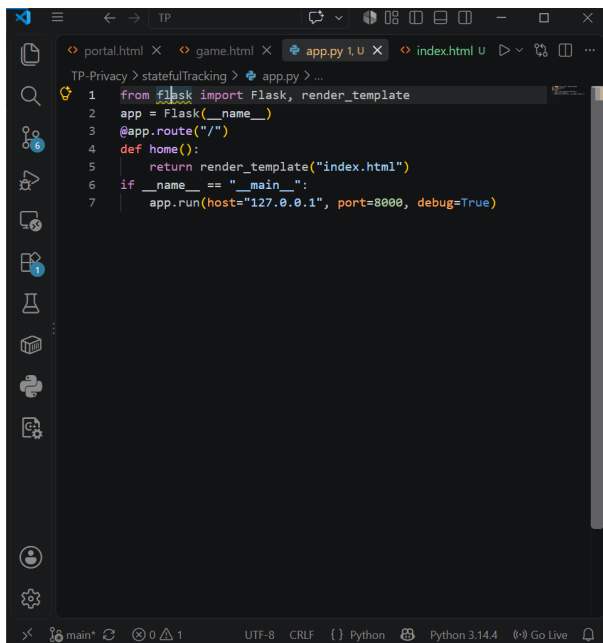
Ahmed Khalil El Atri

September 2026

1 Environment Setup

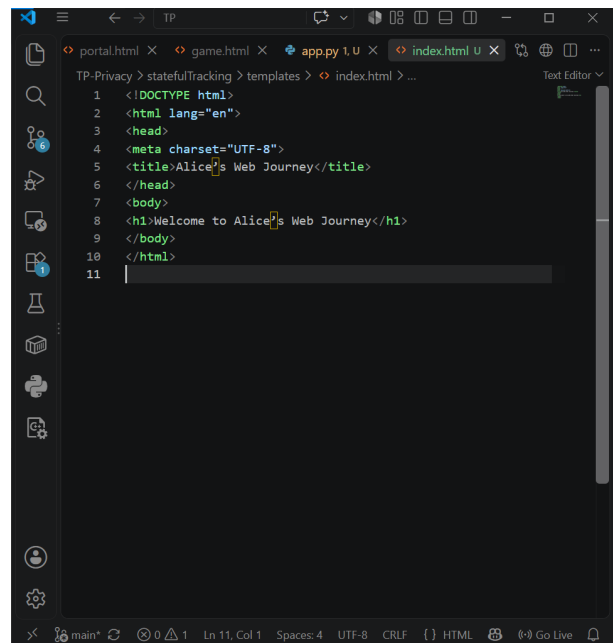
The initial step consisted of preparing an isolated Python environment, configuring a local web server with Flask, and establishing custom local domain resolution.

- Created the project structure with Flask script `app.py` and template `index.html`.
- Initialized and activated a dedicated Python virtual environment (`venv`).
- Installed Flask and verified dependencies.
- Configured the Windows `hosts` file to map `127.0.0.1` to `lab.test`.



```
TP-Privacy > statefulTracking > app.py > ...
1 from flask import Flask, render_template
2 app = Flask(__name__)
3 @app.route("/")
4 def home():
5     return render_template("index.html")
6 if __name__ == "__main__":
7     app.run(host="127.0.0.1", port=8000, debug=True)
```

(a) Flask initial route configuration (`app.py`)



```
TP-Privacy > statefulTracking > templates > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <title>Alice's Web Journey</title>
6 </head>
7 <body>
8 <h1>Welcome to Alice's Web Journey</h1>
9 </body>
10 </html>
11
```

(b) Base template (`templates/index.html`)

Figure 1: Project structure and source code initialization.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/powershell

PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> python --version
Python 3.10.4
PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> python --venv venv
PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> venv\Scripts\activate
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker==1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click==8.1.3 (from flask)
  Downloading click-8.1.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous==2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2==3.1.2 (from flask)
  Downloading jinja2-3.1.2-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe==2.1.1 (from flask)
  Downloading markupsafe-2.1.1-cp310-cp310-win_and64.whl.metadata (2.8 kB)
Collecting werkzeug==3.1.0 (from flask)
  Downloading werkzeug-3.1.0-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.1.3-py3-none-any.whl (125 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.2-py3-none-any.whl (134 kB)
Downloading markupsafe-2.1.1-cp310-cp310-win_and64.whl (15 kB)
Downloading werkzeug-3.1.0-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.1.3 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.2 markupsafe-2.1.1 werkzeug-3.1.0

[notice] A new release of pip is available: 20.0.1 -> 20.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> flask --version
Python 3.10.4
Flask 3.1.3
Werkzeug 3.1.0
PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> python app.py
C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\venv\Scripts\python.exe: can't open file 'C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\app.py': [Error 2] No such file or directory
PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> cd \statefulTracking\
PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> cd \statefulTracking\
PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy> python .\app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PID: 106-333-139

```

(a) PowerShell venv setup and Flask execution

```

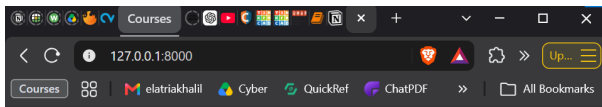
hosts - Notepad
File Edit Format View Help
individual
# lines or following the machine name denoted by a '#'
symbol.
#
# For example:
#
#      102.54.94.97      rhino.acme.com      # source
server
#      38.25.63.10      x.acme.com          # x client
host

# localhost name resolution is handled within DNS itself.
#      127.0.0.1        localhost
#      ::1              localhost
127.0.0.1 lab.test

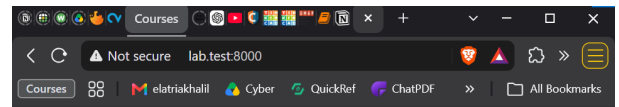
```

(b) Windows hosts file mapping lab.test

Figure 2: Environment creation and hostname mapping.



Welcome to Alice's Web Journey



Welcome to Alice's Web Journey

(a) Initial landing via 127.0.0.1:8000

(b) Landing via alias lab.test:8000

Figure 3: Browser verification of the setup.

2 HTTP Header Inspection

The Flask application was modified to output incoming request headers and outgoing response headers directly to the terminal, and a custom header (X-Lab-Message) was injected.

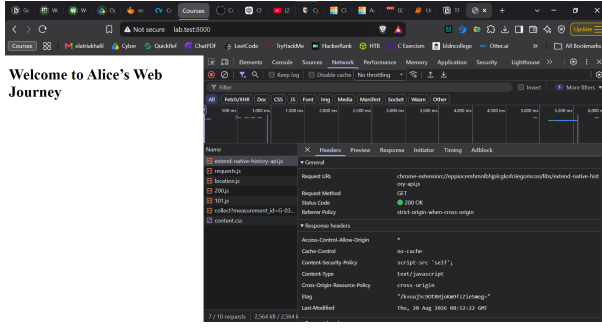
```
response = self.full_dispatch_request()
File "C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\venv\Lib\site-packages\flask\app.py", line 919, in full_d
ispatch_request
rv = self.handle_user_exception(e)
File "C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\venv\Lib\site-packages\flask\app.py", line 917, in full_d
ispatch_request
rv = self.dispatch_request()
File "C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\venv\Lib\site-packages\flask\app.py", line 902, in dispat
ch_request
return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
File "C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\statefulTracking\app.py", line 5, in home
response = make_response(render_template("index.html"))
NameError: name 'make_response' is not defined
127.0.0.1 - - [22/Sep/2026 11:38:38] "GET /?_debugger__=yes&cmd=resource&f=style.css HTTP/1.1" 304 -
127.0.0.1 - - [22/Sep/2026 11:38:38] "GET /?_debugger__=yes&cmd=resource&f=debugger.js HTTP/1.1" 304 -
127.0.0.1 - - [22/Sep/2026 11:38:41] "GET /?_debugger__=yes&cmd=resource&f=console.png HTTP/1.1" 304 -
* Detected change in 'C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\statefulTracking\app.py', reloadi
ng
* Restarting with stat
File "C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\statefulTracking\app.py", line 1
from flask import Flask, render_template,
SyntaxError: trailing comma not allowed without surrounding parentheses
(venv) PS C:\Users\User\Desktop\CC\Info Sec\TP-Privacy\statefulTracking> python .\app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server ins
tead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 108-833-139

--- HTTP Request Headers ---
Host: lab.test:8000
Connection: keep-alive
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Saf
ari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding: gzip, deflate
Accept-Language: fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
Cookie: perf_dv6Tr4n=1

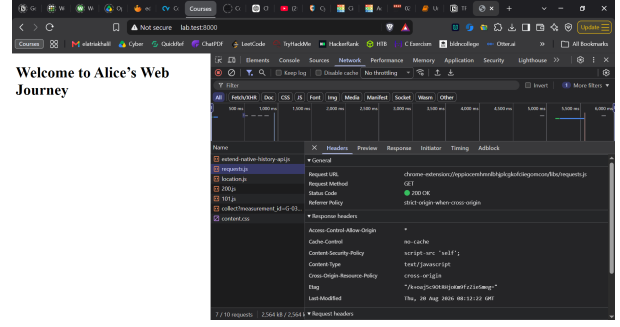
--- HTTP Response Headers ---
Content-Type: text/html; charset=utf-8
Content-Length: 172
X-Lab-Message: Hello from the Flask server

127.0.0.1 - - [22/Sep/2026 11:39:25] "GET / HTTP/1.1" 200 -
```

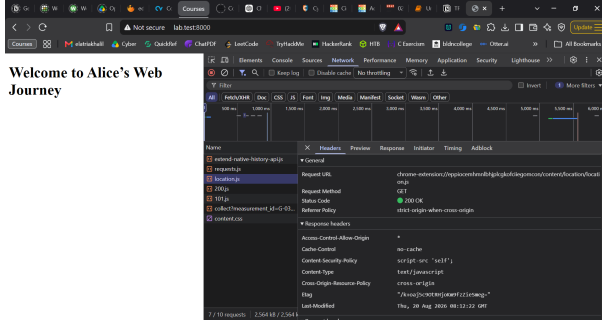
Figure 4: Flask terminal logging HTTP request/response headers and debug trace.



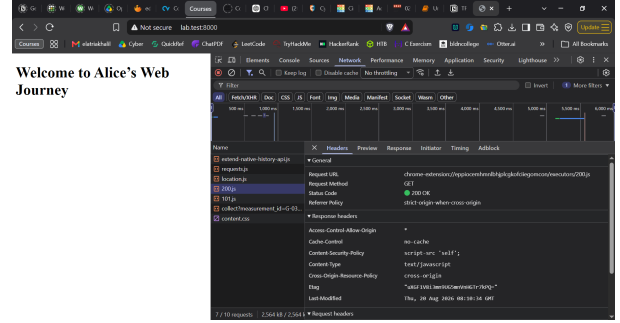
(a) Network inspection: extend-native-history-api.js



(b) Network inspection: requests.js



(c) Network inspection: location.js

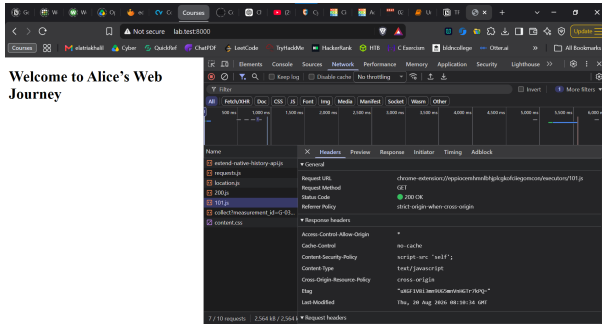


(d) Network inspection: resource execution

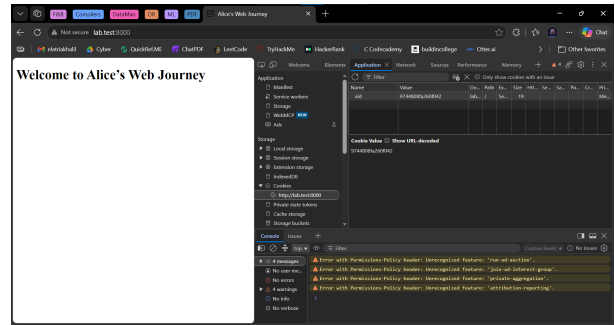
Figure 5: DevTools Network panel: client request and server response header inspection.

3 Cookie Manipulation

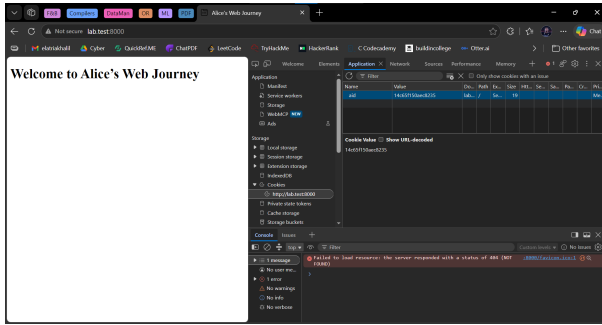
Stateful browser identification was implemented by issuing an identifier cookie (`aid`) via `response.set_cookie()`. Different cookie directives and security flags were evaluated.



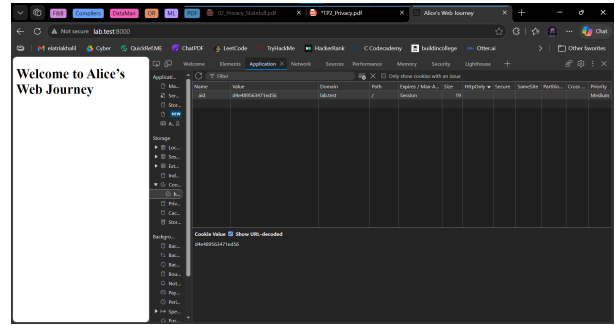
(a) Header inspection during initial cookie handshake



(b) Initial session cookie creation



(c) Cookie renewal test



(d) Inspecting standard session-bound aid

Figure 6: Baseline cookie creation and lifecycle inspection.

3.1 Persistent Cookie

Setting an explicit expiration timestamp transformed the cookie from a session-only artifact to a persistent cookie. The cookie persisted across browser restarts.

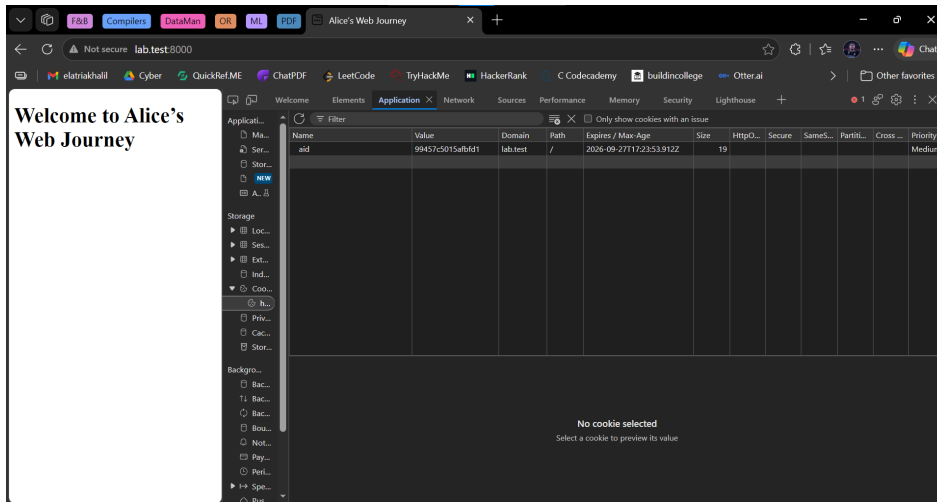
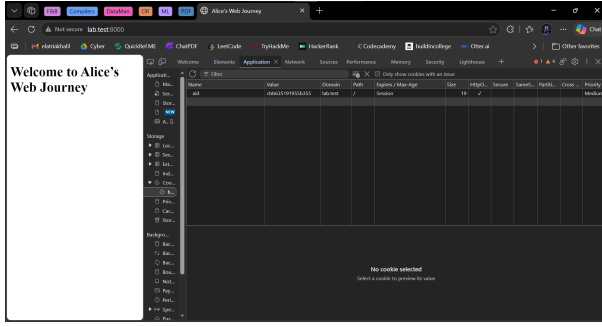


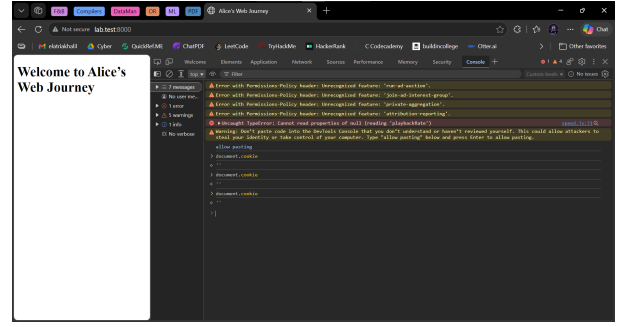
Figure 7: Persistent aid cookie displaying explicit Expires / Max-Age timestamp.

3.2 HttpOnly Flag

Flagging the cookie with `HttpOnly` prevents client-side scripts from reading it. The cookie remains visible to DevTools and is attached to HTTP requests, but querying `document.cookie` in the console evaluates to an empty string.



(a) Application storage displaying checked `HttpOnly` attribute

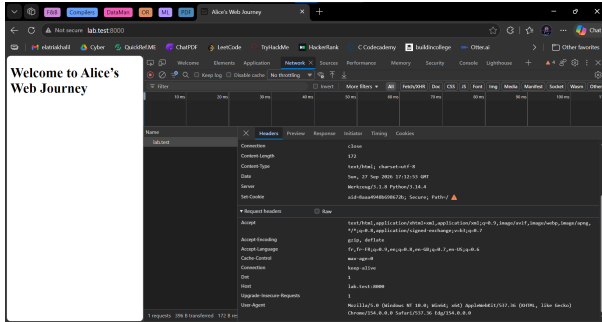


(b) DevTools Console: `document.cookie` returns empty string

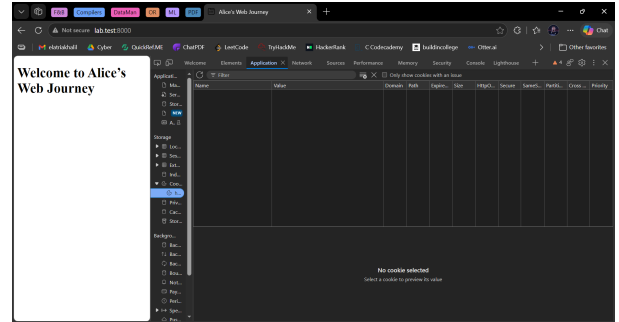
Figure 8: Evaluation of the `HttpOnly` flag against JavaScript access.

3.3 Secure Flag

When the `Secure` attribute is issued over an unencrypted connection (plain HTTP), the browser flags a security violation warning, restricting the storage or usage of the cookie in non-HTTPS contexts.



(a) Network response showing warning icon on `Set-Cookie: aid=...; Secure`

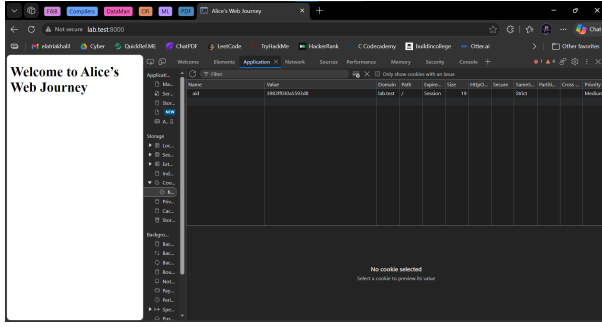


(b) Application tab confirming rejected unencrypted storage

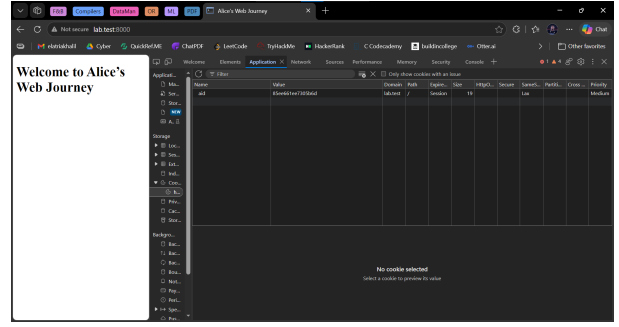
Figure 9: Testing the `Secure` directive over plain HTTP.

3.4 SameSite Flag

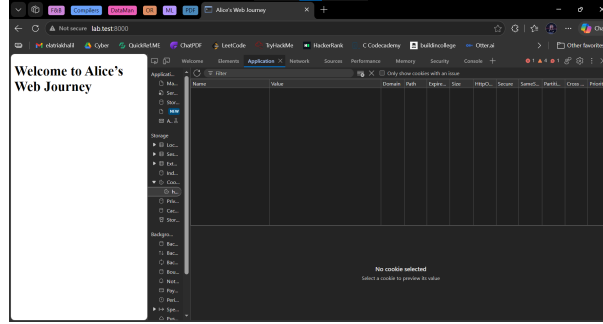
The flags `SameSite=Strict`, `SameSite=Lax`, and `SameSite=None` dictate whether cookies are sent during cross-site requests and top-level navigations.



(a) Cookie configured with SameSite=Strict



(b) Cookie configured with SameSite=Lax

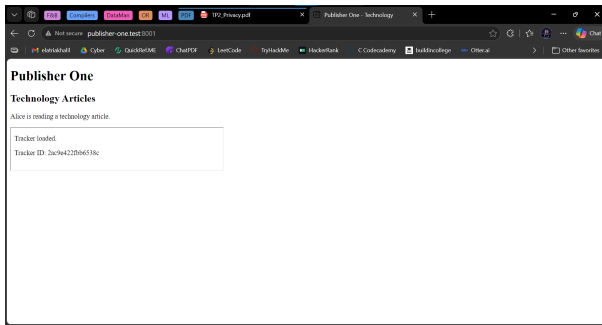


(c) Testing SameSite=None behavior

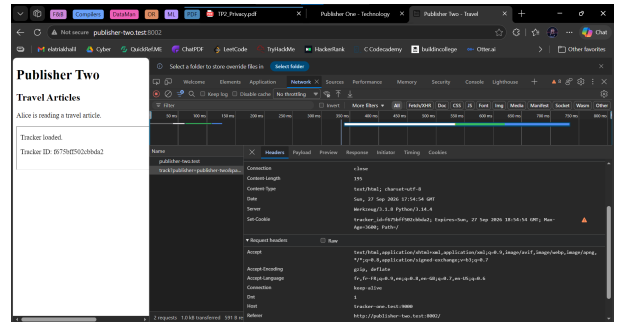
Figure 10: Inspection of SameSite cookie attributes in DevTools.

4 Challenge 1 — Third-Party Tracking

Two publisher platforms (publisher-one.test:8001 and publisher-two.test:8002) were configured to embed an iframe pointing to an independent third-party tracking server (tracker-one.test:9000).

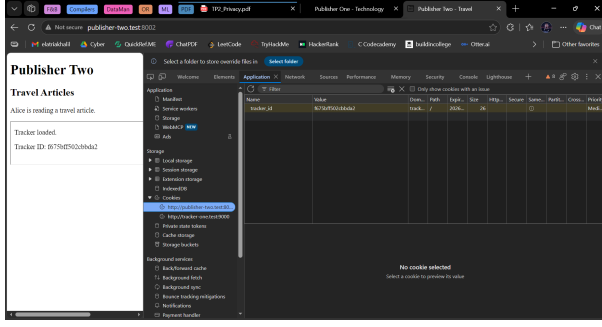


(a) Publisher One embedding tracker iframe

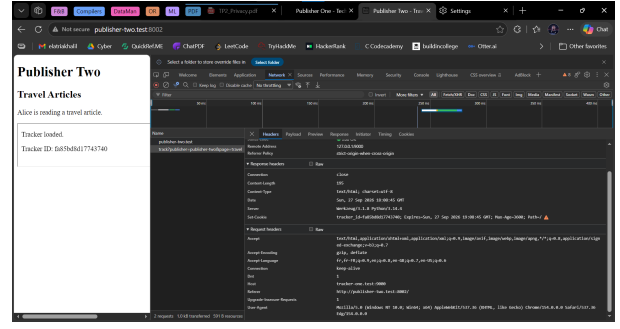


(b) Publisher Two tracker request headers and Set-Cookie

Figure 11: Third-party iframe embedding across independent origins.



(a) Tracker ID stored under Publisher Two context



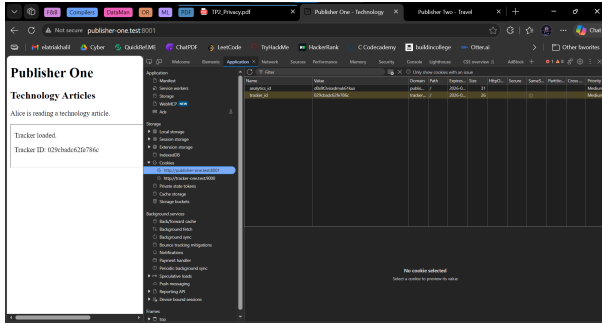
(b) Network panel showing isolated `tracker_id` issuance

Figure 12: Storage and network inspection for third-party cookie handling.

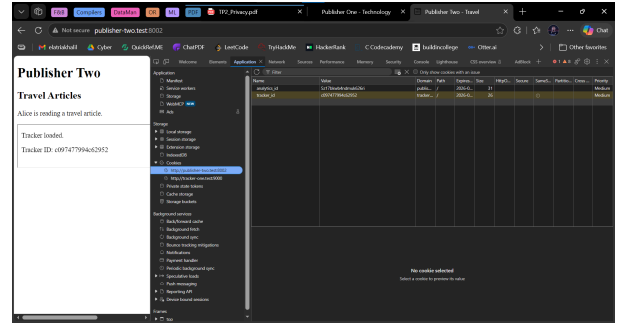
Observations and Finding: Modern browser tracking protections partition third-party storage per top-level site. Consequently, the tracker was unable to link the user under a single shared identifier across both publishers. Instead, separate tracking identifiers were generated for Publisher One and Publisher Two, creating partitioned tracking profiles.

5 Challenge 2 — First-Party Analytics

A dedicated analytics script (`analytics.js`) served from `analytics.test:9100` was integrated into both publisher sites. Because the script executes within the execution context of the host document, the cookies created reside entirely in the first-party domain of each respective publisher.



(a) Publisher One: first-party `analytics_id` alongside tracker cookie



(b) Publisher Two: distinct first-party `analytics_id`

Figure 13: DevTools Application storage showing isolated first-party identifiers per domain.

Observations and Finding:

- `publisher-one.test` stores an isolated first-party identifier (`analytics_id` = `d0x9t3visxdmuk61kus`).
- `publisher-two.test` generates its own separate identifier (`analytics_id` = `5z17bkwb4ndmuk626ri`).
- While the analytics JavaScript can create and access cookies within each publisher page, domain boundaries isolate them. The analytics server receives the identifiers via active API calls (e.g., `/collect`) rather than implicit cross-site cookie transmission, maintaining separate profiles per publisher unless external identity stitching is applied.